

A PROJECT REPORT
ON

Lightweight IoT Intrusion Detection Through Feature Optimization and Explainable Analytics

Project report submitted in partial fulfilment of the requirements for the course

MACHINE LEARNING
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
Academic Year: 2026–27

Submitted by — Team of Two

Dhruva Datta Vishnubhotla	Roll No: 2520039620
A Vishal Goud	Roll No: 2520030521

Under the Esteemed Guidance of

Dr. N. Nirmalajyothi
Associate Professor, CSE



Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Abstract

The rapid expansion of Internet of Things (IoT) deployments has increased the volume, diversity, and security sensitivity of network traffic, making manual intrusion monitoring impractical. This project develops a leakage-aware and explainable machine-learning pipeline for binary IoT network intrusion detection using the IoTID20 flow dataset, which was captured in a controlled smart-home testbed and contains normal traffic together with several attack types. The raw dataset contains 625,783 flows and 86 columns. A systematic audit identified 164,087 exact duplicate rows, non-finite rate values, a malformed destination-port value, constant predictors, high-cardinality identifiers, and target-derived fields that could bias model evaluation. After deduplication, 461,696 unique flows remained. The final test set was isolated before learned preprocessing, feature ranking, or model tuning. Training-only preprocessing produced 72 model-ready features, while a consensus feature-selection strategy combined ANOVA F-test, mutual information, Random Forest importance, and XGBoost importance.

Logistic Regression, Random Forest, and XGBoost were compared using Macro-F1 as the primary metric, supported by balanced accuracy, ROC-AUC, PR-AUC, class-wise measures, model size, and inference time. On the untouched test set of 92,340 flows, Random Forest using only the top 10 features achieved the strongest Macro-F1 of 0.9978 and 99.93% accuracy. XGBoost remained highly competitive with a Macro-F1 of 0.9960, ROC-AUC of 0.9999, and a serialized size of 0.921 MB. SHAP analysis further identified destination port, backward initial window bytes, source port, flow duration, and ACK flag count as dominant decision factors. The study shows that disciplined preprocessing, leakage control, compact feature selection, class-aware evaluation, and explainability can produce a highly accurate and transparent IoT intrusion-detection workflow. Because the data were collected in a controlled environment with a limited number of devices and attacks, further testing on larger real-world IoT networks is required before deployment.

Keywords: *IoT Security; Intrusion Detection; Feature Selection; Random Forest; XGBoost; SHAP.*

PROJECT SNAPSHOT

DATASET IoTID20	TASK Normal vs Anomaly <i>binary classification</i>	RAW FLOWS 625,783	UNIQUE FLOWS 461,696
FINAL PREDICTORS 72	BEST MODEL Random Forest <i>top 10 features</i>	TEST MACRO-F1 0.9978	EXPLAINABILITY SHAP

Table of Contents

Abstract..... 2

1. Introduction..... 5

1.1 Purpose of the Project..... 5

1.2 Problem Statement..... 5

1.3 Existing Methods and Disadvantages..... 5

1.4 Proposed System..... 6

1.5 Research Objectives..... 6

2. Literature Survey (Recent Studies: 2023–2025)..... 7

3. Methodology..... 11

3.1 Proposed System Flow Diagram..... 11

3.2 Dataset Description..... 12

3.3 Experimental Setup..... 13

3.4 Data Preprocessing..... 14

3.5 Exploratory Data Analysis..... 15

3.6 Feature Engineering and Feature Selection..... 15

3.7 Model Building and Training..... 17

3.8 Hyperparameter Tuning..... 17

3.9 Model Evaluation..... 19

4. Results and Discussion..... 20

4.1 Preprocessing Results..... 20

4.2 Model Performance..... 20

4.3 Visual Results and Discussion..... 22

4.4 Discussion of Results..... 24

5. Conclusion..... 25

6. Future Work..... 26

7. References..... 26

List of Figures

Figure 1. Proposed leakage-aware ML workflow.....	11
Figure 2. Class distribution after duplicate removal.....	13
Figure 3. Consensus feature ranking (top 20).....	17
Figure 4. XGBoost validation Macro-F1 by feature count.....	19
Figure 5. Final test-set Macro-F1 by model.....	21
Figure 6. Random Forest confusion matrix (final test).....	22
Figure 7. SHAP mean absolute impact (Random Forest).....	24

List of Tables

Table 1. Summary of recent IoT intrusion-detection studies.....	7
Table 2. Stages of the proposed workflow.....	12
Table 3. Dataset statistics.....	13
Table 4. Software environment.....	14
Table 5. Exploratory analyses and their purpose.....	15
Table 6. Excluded columns.....	16
Table 7. Top ten consensus-ranked features.....	16
Table 8. Final model configurations.....	18
Table 9. XGBoost baseline versus tuned parameters.....	18
Table 10. XGBoost validation Macro-F1 by feature count.....	18
Table 11. Evaluation metrics and their role.....	19
Table 12. Final test-set model performance.....	20
Table 13. Validation versus test Macro-F1.....	21
Table 14. XGBoost validation and test metrics.....	23
Table 15. Mean absolute SHAP value by feature.....	23

1. Introduction

1.1 Purpose of the Project

IoT ecosystems connect sensors, appliances, gateways, embedded controllers, and cloud services at a scale that makes continuous human inspection impossible. Smart homes, industrial systems, healthcare devices, and transportation systems all generate growing volumes of network traffic. These devices frequently operate with limited computing resources and diverse software stacks, while still exchanging security-sensitive data. As connectivity grows, anomalous behavior such as unauthorized access, scanning, denial-of-service activity, brute-force access, command-and-control communication, and botnet traffic can disrupt availability or compromise device integrity. The practical stakeholders include network administrators, security operations teams, IoT platform owners, service providers, researchers in network security, and end users who depend on reliable connected services.

The project therefore investigates whether a carefully engineered machine-learning system can distinguish Normal from Anomaly traffic with high class-balanced performance while remaining compact enough to understand and potentially deploy near the network edge. The system aims to reduce unnecessary features and computational requirements while maintaining strong classification performance, and it includes explainability so that the network characteristics driving each decision can be analyzed rather than treating the model as a black box. The emphasis is not only on obtaining a high benchmark score, but on building a credible workflow: duplicated data are removed, target leakage is controlled, transformations are fitted on training data only, multiple feature-ranking criteria are combined, candidate models are tuned using validation data, and the final test set is evaluated only after model decisions are frozen.

1.2 Problem Statement

Given a network-flow record from IoTID20, the system must perform binary classification and predict whether the flow is Normal (0) or Anomaly (1). The input consists of numerical flow characteristics such as ports, packet and byte statistics, inter-arrival times, TCP flags, rate features, active/idle intervals, and window information. The expected output is a binary class label together with prediction probabilities that support evaluation using accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC, as well as feature-level model explanations. Because the dataset is strongly imbalanced toward anomalous traffic, the system must avoid relying on overall accuracy alone and must explicitly measure performance on both classes.

1.3 Existing Methods and Disadvantages

Traditional intrusion detection commonly uses signature-based systems, manually defined rules, statistical techniques, or conventional machine-learning models. Signature-based detection is effective when an attack pattern is already known and represented by a rule or signature, but its coverage can degrade when traffic behavior changes or when attacks do not closely match existing signatures. Machine-learning approaches can learn patterns directly from network data, yet their performance depends strongly on data quality, feature selection, class imbalance, and model configuration. Some reported IDS pipelines can also be optimistic if they split duplicated data, retain identifiers that enable memorization, select features using all samples, or tune models after observing the final test results. Using a large number of unnecessary features further increases computational requirements and makes deployment harder.

A second limitation is the trade-off between predictive power, computational cost, and interpretability. Linear classifiers are efficient and transparent but may miss complex non-linear interactions. High-performing ensemble and deep models can capture richer relationships but may increase model size, inference cost, and black-box behavior. In addition, evaluating models only with overall accuracy can be misleading when the classes are highly imbalanced. Recent studies increasingly address these weaknesses through feature reduction [1], [10]–[17], edge-aware evaluation [8], [9], and Explainable AI (XAI) methods

such as SHAP and LIME [3]–[5], [7], [9], [10], [13], [15]. A suitable system should therefore use multiple evaluation metrics, prevent data leakage during preprocessing and feature selection, reduce unnecessary features where possible, and explain its predictions. The present project follows this direction while focusing on classical ensemble models that are well suited to structured network-flow data.

1.4 Proposed System

The proposed system is an end-to-end, leakage-aware pipeline that begins with data auditing and exact-duplicate removal, excludes target-derived and identifier fields, isolates an untouched final test set, performs training-fitted imputation and scaling, and ranks features through a four-method consensus. Feature subsets of 10, 15, 20, 30, and all 72 features are compared to study the effect of feature optimization, and Logistic Regression, Random Forest, and XGBoost candidates are trained with class-aware weighting. Hyperparameters and feature counts are selected on validation data only. The frozen models are then evaluated once on the final test split using Macro-F1, precision, recall, ROC-AUC, and PR-AUC, and SHAP is used to interpret the strongest candidate. The contribution is therefore a combination of data-quality control, feature optimization, robust evaluation, and explainability rather than a single classifier in isolation.

1.5 Research Objectives

1. Prepare a reliable IoTID20 dataset by auditing the data, correcting schema issues, removing exact duplicates, treating non-finite values, defining the Normal and Anomaly classes, and excluding information that can cause target or identifier leakage.
2. Perform systematic preprocessing and exploratory data analysis to understand class balance, feature behavior, correlations, traffic distributions, and data-quality risks before model training.
3. Engineer and select relevant features while preventing data leakage: every selection procedure is learned from training data only, and different feature subsets are compared.
4. Develop and tune Logistic Regression, Random Forest, and XGBoost models for binary IoT intrusion classification.
5. Evaluate and compare the models using accuracy, precision, recall, Macro-F1, ROC-AUC, PR-AUC, confusion matrices, and other class-sensitive performance measures.
6. Identify the best-performing model from validation results and assess its practical characteristics, including feature requirements, model size, computational considerations, and predictive performance.
7. Apply SHAP-based explainable analytics to identify important features and understand how individual network characteristics contribute to the model's predictions.
8. Evaluate the finalized models on an untouched test set to obtain an unbiased estimate of their performance on previously unseen IoT network traffic.

2. Literature Survey (Recent Studies: 2023–2025)

The literature survey reviews seventeen recent studies that are directly relevant to IoT/network intrusion detection, feature selection and reduction, ensemble learning, edge/TinyML deployment, class imbalance, and explainability. Each entry gives the bibliographic reference with its DOI. Numerical results are not treated as directly comparable to this project because datasets, splits, preprocessing rules, target granularity, and evaluation protocols differ across studies.

Table 1. Summary of recent studies on IoT intrusion detection (2023–2025).

S.No.	Title	Reference with DOI	Problem / Gap	Research Objective	Methods / Focus	Key Inference	Future Research
1	Machine Learning Algorithms for Intrusion Detection in IoT Prediction and Performance Analysis	E. El Mahfoud, S. El Hajla, Y. Maleh, S. Mounir; <i>Procedia Computer Science</i> , 236, 460–467 (2024). DOI: 10.1016/j.procs.2024.05.054	Need to compare classical ML choices on IoTID20 while reducing unnecessary features and execution cost.	Evaluate multiple ML algorithms for IoT intrusion detection and identify an effective predictive approach.	IoTID20; DT, RF, k-NN, AdaBoost, SVM; feature selection; accuracy/error comparison.	Decision Tree reported the strongest overall accuracy (~99.80%) in the study; feature reduction supported efficient prediction.	Validate with stricter temporal/grouped splits, external datasets, richer class-wise metrics, and explainability.
2	A comparative analysis of various machine learning methods for anomaly detection in cyber attacks on IoT networks	M. M. Inuwa, R. Das; <i>Internet of Things</i> , 26, 101162 (2024). DOI: 10.1016/j.iot.2024.101162	Algorithm behavior can vary across heterogeneous IoT datasets; practitioners need comparative evidence rather than one-model assumptions.	Compare supervised ML methods for anomaly detection and identify strengths across IoT cyberattack scenarios.	ToN-IoT and BoT-IoT; SVM, ANN, DT, Logistic Regression, k-NN; multi-criteria comparison.	The neural-network approach performed best overall in the reported experiments, showing that model choice depends on traffic structure.	Study cross-network transfer, feature robustness, interpretable ensembles, and resource-constrained deployment.
3	Explainable AI-based intrusion detection in IoT systems	S. Bin Hulayyil, S. Li, N. Saxena; <i>Internet of Things</i> , 31, 101589 (2025). DOI: 10.1016/j.iot.2025.101589	ML-based IDS can suffer from false positives, changing attacks, data bias, and low transparency.	Develop an explainable IDS framework that detects vulnerable IoT devices and makes model decisions understandable.	CUSmartHome + IoT23/other data; ML classifiers; SHAP and causal/explainability analysis; binary and multiclass evaluation.	Combining predictive models with XAI improved the interpretability of vulnerability and intrusion decisions across multiple datasets.	Extend to online adaptation, unseen vulnerabilities, additional devices, and long-term real-network validation.
4	Enhancing AI transparency in IoT intrusion detection using explainable AI techniques	Y. Wang, M. A. Azad, M. Zafar, A. Gul; <i>Internet of Things</i> , 33, 101714 (2025). DOI: 10.1016/j.iot.2025.101714	Accurate AI-driven IDS may be difficult for non-experts to trust when decision logic is opaque.	Quantify how SHAP and LIME can improve transparency for interpretable ML models in IoT intrusion detection.	Decision Tree and Logistic Regression; SHAP; LIME; LIAM framework; visualization and dimensionality-reduction support.	XAI added meaningful feature-level explanations even to relatively transparent models and supported stakeholder understanding.	Evaluate explanation quality with users, stronger classifiers, multiple datasets, and operational SOC workflows.
5	IoT-based intrusion detection system using explainable multi-class deep learning approaches	S. Sadhwani, A. Navare, A. Mohan, R. Muthalagu, P. M. Pawar; <i>Computers and Electrical Engineering</i> , 123, 110256 (2025). DOI: 10.1016/j.compeleceng.2025.110256	Deep IDS can be accurate but computationally expensive and difficult to interpret across multiple attack classes.	Create explainable multiclass DL IDS models and use explanations to identify a reduced high-impact feature subset.	NSL-KDD, UNSW-NB15, TON-IoT, X-IIoTID; CNN, LSTM, Bi-LSTM; SHAP; 15-feature reduction.	SHAP-guided reduced feature sets preserved strong performance while improving training efficiency and interpretability.	Test streaming/edge deployment, cross-dataset transfer, concept drift, and explanation stability over time.

S.No.	Title	Reference with DOI	Problem / Gap	Research Objective	Methods / Focus	Key Inference	Future Research
6	NIDS-CNNRF integrating CNN and random forest for efficient network intrusion detection model	K. Yang et al.; <i>Internet of Things</i> , 32, 101607 (2025). DOI: 10.1016/j.iot.2025.101607	High-dimensional traffic, class imbalance, and emphasis on aggregate accuracy can weaken multiclass IDS quality.	Improve feature extraction, class balance, and attack-type discrimination in network intrusion detection.	CNN feature extraction + Random Forest; ADASYN; PCA; evaluation on KDD CUP99, NSL-KDD, CIC-IDS2017, CIC-IDS2018.	The hybrid CNN-RF design improved reported detection quality by combining learned representations, resampling, and ensemble classification.	Evaluate on modern IoT-native traffic, reduce deployment cost, and analyze robustness against distribution shift.
7	An interpretable deep learning framework for intrusion detection in industrial Internet of Things	J. Ahmad et al.; <i>Internet of Things</i> , 33, 101681 (2025). DOI: 10.1016/j.iot.2025.101681	IIoT IDS require high detection performance while retaining transparent reasoning in safety-critical industrial environments.	Build an interpretable hybrid IDS that selects influential inputs and captures temporal dependencies.	Extra Trees feature selection + GRU (XGRU-IDS); CICIoT2023; SHAP summary/importance/force explanations.	The framework reported 97.56% accuracy and demonstrated that interpretability can coexist with strong multiclass detection.	Benchmark on live industrial streams, evaluate latency/energy on edge hardware, and monitor explanation drift.
8	Evaluating the performance of TinyML singular and ensemble techniques for intrusion detection in IoT networks	A. Hamdouchi, A. Idri; <i>Microprocessors and Microsystems</i> , 117, 105172 (2025). DOI: 10.1016/j.micpro.2025.105172	Cloud inference adds latency and may be unsuitable for constrained IoT nodes; model quality must be considered with RAM/flash/latency.	Compare singular and ensemble TinyML IDS configurations under real microcontroller resource constraints.	NF-ToN-IoT-v2 and NF-BoT-IoT-v2; RF, XGBoost, Extra Trees, DT, Gaussian NB, MLP; mRMR/ANOVA; Arduino UNO.	Resource-aware ranking showed that the best predictive architecture is not always the best embedded choice; singular models were competitive.	Expand to additional microcontrollers, streaming traffic, energy measurement, and adaptive/online TinyML models.
9	Real-time explainable IoT security with machine learning and CTGAN-enhanced detection for resource-constrained devices	T. Hasan, S. Tasnim; <i>Ad Hoc Networks</i> , 178, 103937 (2025). DOI: 10.1016/j.adhoc.2025.103937	IDS datasets are often imbalanced and accurate models may lack explainability; real-time deployment adds latency and memory constraints.	Create an interpretable, imbalance-aware IDS and validate its practical edge-device behavior.	CTGAN for minority-data generation; ML classifiers; SHAP + LIME; multi-dataset evaluation; Jetson Nano deployment.	The work linked predictive results with model size, test time, and explainability, providing a more deployment-oriented evaluation.	Assess long-term streaming drift, adversarial robustness, automatic retraining, and additional low-power edge devices.
10	Intrusion detection in the Internet of Things using convolutional neural networks: an explainable AI approach	F. Ebrahimi, R. Javidan, R. Akbari, Y. Hosseini; <i>Cybersecurity</i> , 8, Article 66 (2025). DOI: 10.1186/s42400-025-00369-2	Complex IDS models can be difficult to interpret and too heavy for resource-constrained IoT environments.	Develop a lightweight 1D-CNN IDS with feature reduction and model-agnostic explanation support.	TON-IoT; SHAP-based feature selection; lightweight 1D-CNN; SHAP and LIME explanations.	The study reported approximately 0.995 accuracy and showed that reduced-feature models can retain strong performance while improving transparency.	Evaluate on physical IoT gateways, additional attack families, temporal splits, and explanation robustness under unseen traffic.
11	Designing accurate lightweight intrusion detection systems for IoT networks using fine-tuned linear SVM and feature selectors	J. Azimjonov, T. Kim; <i>Computers & Security</i> , 137, 103598 (2024). DOI: 10.1016/j.cose.2023.103598	Conventional IDS approaches can become computationally expensive when all available features are used, particularly on resource-constrained IoT devices.	Develop a lightweight and accurate IoT IDS by combining feature selection with a fine-tuned Linear SVM.	Importance-coefficient, forward sequential, backward sequential, and correlation-coefficient feature selection; Linear SVM; KDD Cup 1999, Bot-IoT 2018, and N-BaIoT 2021 datasets.	A substantially reduced feature set (six features) maintained strong detection performance while lowering the feature-processing requirement.	Validate the selected features on newer heterogeneous IoT datasets and real resource-constrained edge devices, and incorporate XAI for interpretable decisions.

S.No.	Title	Reference with DOI	Problem / Gap	Research Objective	Methods / Focus	Key Inference	Future Research
12	Empirical evaluation of feature selection methods for machine learning based intrusion detection in IoT scenarios	J. García, J. Entrena, Á. Alesanco; <i>Internet of Things</i> , 28, 101367 (2024). DOI: 10.1016/j.iot.2024.101367	IoT IDSs may contain redundant or irrelevant attributes, increasing computational requirements and potentially affecting classification.	Systematically evaluate different feature-selection approaches with ML classifiers for IoT intrusion-traffic classification.	Filter feature ranking (FFR), filter feature-subset selection (FSS), and wrapper feature selection (WFS); multiple ML classifiers across four IoT intrusion datasets.	Feature selection reduced the number of attributes while generally preserving performance; F1-scores above 0.99 were reported in most experiments with attribute reductions exceeding 60%.	Investigate feature stability across datasets, real-time feature-extraction costs, and deployment on heterogeneous edge/IoT platforms.
13	An explainable efficient flow-based Industrial IoT intrusion detection system	M. M. Alani; <i>Computers & Electrical Engineering</i> , 108, 108732 (2023). DOI: 10.1016/j.compeleceng.2023.108732	Many ML-based IIoT IDSs operate as black boxes and may use unnecessarily large feature sets.	Develop an efficient and explainable IIoT intrusion-detection system using a compact network-flow feature set.	Recursive Feature Elimination (RFE); ML-based IDS; SHAP; WUSTL-IIoT-2021 dataset; feature set reduced to 11 features.	Feature reduction and SHAP-based explanation can be combined in an IIoT IDS; the reported system achieved 99.97% accuracy using 11 features.	Extend evaluation to heterogeneous IoT environments, newer attacks, real-time traffic, and resource-constrained hardware.
14	Approach for detecting attacks on IoT networks based on ensemble feature selection and deep learning models	S. D. A. Rihan, M. Anbar, B. A. Alabsi; <i>Sensors</i> , 23(17), 7342 (2023). DOI: 10.3390/s23177342	Individual feature-selection methods may retain redundant features, while deep-learning IDSs can be computationally demanding.	Combine multiple feature-selection techniques and RFE with deep-learning models for IoT attack detection.	Variance threshold, mutual information, chi-square, ANOVA, and L1-based selection followed by RFE; CNN, RNN, GRU, and LSTM; IoT-Botnet 2020 dataset.	Combining filter methods and RFE produced a compact feature set and supported high detection performance across the evaluated deep-learning models.	Evaluate on more recent and diverse IoT datasets, investigate lightweight models for edge deployment, and assess computational overhead during real-time operation.
15	Explainable deep learning-based feature selection and intrusion detection method on the Internet of Things	X. Chen, M. Liu, Z. Wang, Y. Wang; <i>Sensors</i> , 24(16), 5223 (2024). DOI: 10.3390/s24165223	Deep-learning IDSs can have high computational complexity and limited interpretability, creating difficulties for practical deployment.	Develop an interpretable feature-selection mechanism that uses model explanations to reduce features while maintaining detection reliability.	SHAP and causality-based feature selection; CNN and Random Forest; CICIDS2017 and NSL-KDD datasets.	SHAP-based interpretation can be built into feature selection, linking explainability with dimensionality reduction rather than treating them as separate stages.	Validate SHAP-guided feature selection on IoT-specific and real-time datasets and investigate its computational cost for edge deployment.
16	Optimizing IoT intrusion detection using balanced class distribution, feature selection, and ensemble machine learning techniques	M. B. Mustafa et al.; <i>Sensors</i> , 24(13), 4293 (2024). DOI: 10.3390/s24134293	Class imbalance and irrelevant features can contribute to poor IDS performance and overfitting.	Optimize IoT intrusion detection through class balancing, feature selection, and ensemble learning.	Class balancing and ANOVA feature selection; SVM with bagging and LSTM with stacking; UNSW-NB15 and NSL-KDD datasets.	Class balancing, feature selection, and ensemble learning were combined to counter imbalance and irrelevant-feature effects, and feature relationships linked to particular attack patterns were identified.	Investigate more realistic IoT datasets, leakage-safe evaluation procedures, and lightweight ensemble models for deployment.

S.No.	Title	Reference with DOI	Problem / Gap	Research Objective	Methods / Focus	Key Inference	Future Research
17	Enhancing IoT network security using feature selection for intrusion detection systems	M. Almohaimeed, F. Albalwy; <i>Applied Sciences</i> , 14(24), 11966 (2024). DOI: 10.3390/app142411966	IoT traffic contains many attributes, and redundant features can increase model complexity and reduce practical efficiency.	Investigate feature-selection techniques for improving IoT intrusion detection and identifying important network attributes.	Information gain, gain ratio, correlation-based feature selection, Pearson correlation, and symmetric uncertainty; RT-IoT2022 dataset with 83 input features.	The study identifies feature relationships associated with particular attack patterns and shows how feature selection can support IoT IDS development.	Evaluate selected features across additional IoT datasets and integrate them with lightweight classifiers and explainable models.

Literature synthesis: across the recent evidence, four themes repeatedly emerge: (1) feature reduction can lower cost without necessarily reducing detection quality [1], [5], [10]–[17]; (2) class imbalance and leakage-aware evaluation matter as much as the classifier choice [6], [9], [16]; (3) explainability, increasingly SHAP, is used both to interpret models and to guide feature selection [3]–[5], [7], [10], [13], [15]; and (4) device-level resource measurements are becoming essential for trustworthy IoT IDS deployment [8], [9]. This project addresses these themes directly using training-only feature selection, Macro-F1-driven comparison, final-test isolation, model-size and inference measurements, and SHAP explanations.

3. Methodology

3.1 Proposed System Flow Diagram

Figure 1 summarizes the workflow and Table 2 describes each of its stages. The train–validation–test split is performed before any learned transformation, so every fitted step (imputation, scaling, feature ranking, tuning) sees training or validation data only, and the final test set stays isolated until the candidate models are frozen.

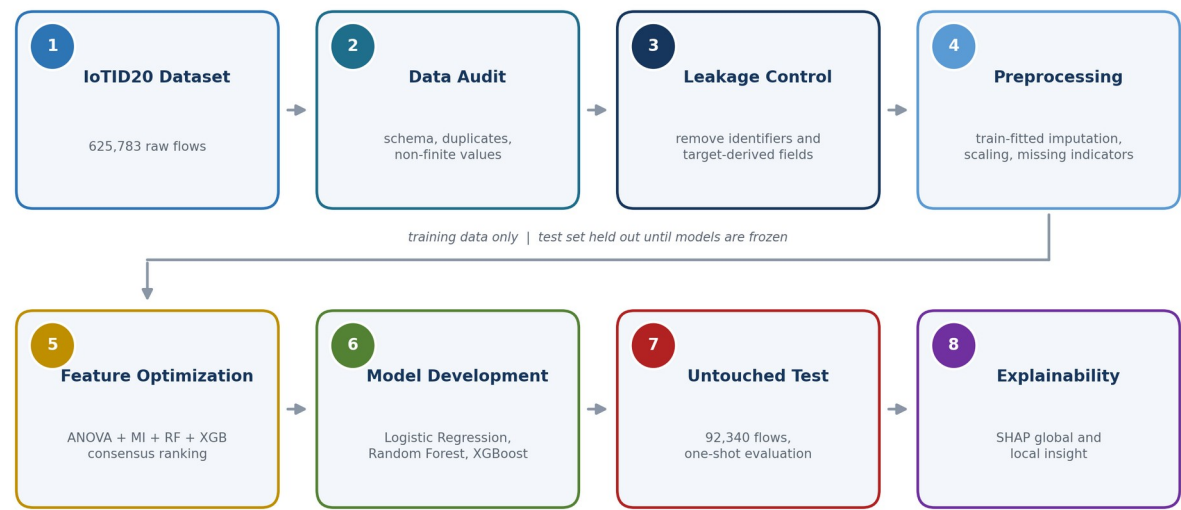


Figure 1. Proposed leakage-aware machine-learning workflow, from raw IoTID20 flows to final explanation.

Table 2. Stages of the proposed workflow.

Stage	Step	What happens
1	Dataset	IoTID20 flow records: 625,783 rows $\times$ 86 columns.
2	Data inspection and schema validation	Dimensions, column names, data types, missing values, duplicates, unique values, and the <code>Label</code> distribution are checked; the malformed <code>Dst_Port</code> value and non-finite rate values are flagged.
3	Duplicate removal	164,087 exact duplicate rows are removed, leaving 461,696 unique flows.
4	Train–validation–test split	The final test set (92,340 flows) is isolated before any learned step; 295,484 flows are used for training and 73,872 for validation.
5	Missing-value analysis and treatment	Infinite values become missing values; median imputation and missingness indicators are fitted on training data only.
6	Categorical encoding / data-type handling	Target-derived and identifier columns are excluded; all remaining predictors are numeric, so no one-hot encoding is required.
7	Feature engineering and selection	Four-method consensus ranking computed on training data; fixed top-10, 15, 20, 30, and full 72-feature subsets are saved.
8	Class-aware model training	Logistic Regression, Random Forest, and XGBoost are trained with class weights or sample weights.
9	Hyperparameter tuning	Configurations and feature counts are chosen on validation Macro-F1 only.
10	Prediction	Frozen models produce class labels and class probabilities.
11	Model evaluation	Macro-F1, accuracy, balanced accuracy, ROC-AUC, PR-AUC, per-class metrics, confusion matrix, model size, and inference time.
12	SHAP explainability analysis	Global feature attribution for the selected Random Forest model.
13	Final model and results	The strongest model is reported with its untouched test-set evidence.

3.2 Dataset Description

The project uses IoTID20, a network-flow dataset designed for anomalous activity research in IoT environments [18] and referred to in the project notebooks as the Final IoT Network Intrusion Dataset. The loaded source file contains 625,783 rows and 86 columns. The target variable is `Label`, where Normal is mapped to 0 and Anomaly to 1. Three fields are target-related (`Label`, `Cat`, `Sub_Cat`), four are high-cardinality identifiers or metadata (`Flow_ID`, `Src_IP`, `Dst_IP`, `Timestamp`), and the remaining columns describe flow behavior through ports, protocols, packet/byte counts, flow duration, inter-arrival statistics, TCP flag counts, flow rates, active/idle periods, and TCP window properties. After removing 164,087 exact duplicates, 461,696 unique flows remain: 423,098 Anomaly records (91.64%) and 38,598 Normal records (8.36%). Table 3 summarizes the dataset.

According to the dataset description, the traffic was generated in a smart-home environment containing IoT devices such as the SKT NUGU (NU 100) smart speaker and the EZVIZ Wi-Fi camera (C2C Mini O Plus 1080P), with laptops and smartphones connected to the same wireless network. The data consist of 42 raw packet-capture (PCAP) files collected at different time intervals using the monitor mode of a wireless network adapter, with unnecessary wireless headers removed using Aircrack-ng. Besides normal traffic, the dataset contains several malicious activities, most of them created with commonly available tools such as Nmap; for the Mirai botnet attack, malicious packets were first generated on a laptop and then modified to

appear as if they originated from an IoT device. Because the dataset was created in a controlled environment with a limited number of devices and attacks, the results should be read within that scope (see Section 4.4).

Table 3. Dataset statistics for IoTID20 before and after preprocessing.

Dataset statistic	Value
Raw records	625,783
Raw columns	86
Target variable	Label (0 = Normal, 1 = Anomaly)
Exact duplicates removed	164,087 (26.22%)
Unique records after deduplication	461,696
Anomaly class	423,098 (91.64%)
Normal class	38,598 (8.36%)
Target, identifier and metadata columns excluded	7
Final processed feature count	72
Train / Validation / Test	295,484 / 73,872 / 92,340
Random state	42

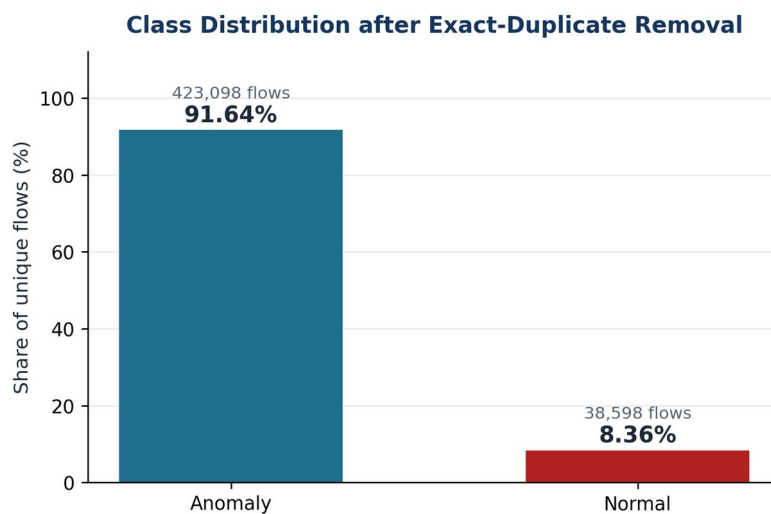


Figure 2. Binary class distribution after exact-duplicate removal; the Anomaly class represents 91.64% of unique flows.

3.3 Experimental Setup

All project notebooks were executed in a Jupyter/IPython workflow using Python 3.14.4, as recorded in the notebook metadata. A fixed random state of 42 was used wherever reproducibility was required. The experiments are organized into separate notebooks covering data preparation, exploratory data analysis, feature selection, model development, XGBoost, explainability, and final evaluation. The notebooks do not persist the exact CPU, RAM, or operating-system specification; therefore, measured inference times are reported as environment-specific observations rather than universal latency guarantees.

Table 4. Software environment and libraries.

Purpose	Tool / setting
Language and workflow	Python 3.14.4 in a Jupyter/IPython notebook environment
Data handling	pandas, NumPy
Preprocessing, feature selection, evaluation, Logistic Regression and Random Forest	scikit-learn
Gradient boosting	XGBoost
Explainability	SHAP
Visualization	Matplotlib, Seaborn
Model persistence	Joblib (frozen models reloaded for final evaluation and explanation)
Reproducibility	Fixed random state of 42

3.4 Data Preprocessing

3.4.1 Data inspection and schema validation

The dataset was first inspected for its dimensions, column names, data types, missing values, duplicate records, unique values, and target-label distribution. The audit identified a malformed `Dst_Port` value, exact duplicates, non-finite values in `Flow_Byts/s` and `Flow_Pkts/s`, and constant predictors. The target variable was verified as `Label`, with Normal and Anomaly as the two binary classes.

3.4.2 Duplicate removal

A total of 164,087 exact duplicate rows (26.22% of the raw data) were removed before model development. Repeated observations can bias the learning process and, when they fall on both sides of a split, cause overly optimistic train–test similarity and inflated evaluation results.

3.4.3 Missing-value analysis and treatment

Missing values were examined for each feature, with numerical and categorical variables checked separately. Infinite values were converted to missing values, after which numeric median imputation was fitted using training data only; missingness indicators were retained for affected variables where appropriate. The pipeline guarantees that no transformation learned from the training data was fitted using validation or test observations.

3.4.4 Outlier analysis

Numerical variables were examined with statistical summaries and visualizations such as box plots. Because extreme values can represent legitimate network behavior (for example, very long flows or bursts during a scan), outliers were not deleted automatically. Treatment was applied only where justified by the characteristics of the feature and its role in intrusion detection.

3.4.5 Categorical encoding and leakage control

`Cat` and `Sub_Cat` were removed because they are alternate target descriptions, and `Flow_ID`, `Src_IP`, `Dst_IP`, and `Timestamp` were removed to avoid high-cardinality encodings that would encourage memorization of individual flows, devices, or capture moments. Constant features were also removed. All remaining predictors were numerical, so no one-hot encoding was required.

3.4.6 Numerical scaling

A standardized representation was created for scale-sensitive, coefficient-based models such as Logistic Regression. Imputed but unscaled features were retained for the tree ensembles, because Random Forest and XGBoost split on feature thresholds and do not require standardization.

3.4.7 Target-label preparation

The `Label` column was converted into a binary target: `0` for Normal traffic and `1` for Anomaly traffic.

3.4.8 Train, validation, and test separation

The data were divided into approximately 64% training (295,484 flows), 16% validation (73,872 flows), and 20% final test (92,340 flows) portions. The final test set was isolated first and remained untouched until the final evaluation. Learned preprocessing, feature ranking, feature-count selection, and hyperparameter tuning were restricted to the training and validation data, which removes the main route for data leakage.

3.5 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the structure and behavior of the network-flow data before model development. It examined dataset dimensions, data types, missing values, duplicates, descriptive statistics, class-wise behavior, correlations, and temporal/protocol characteristics. Table 5 lists the analyses and what each was used for.

Table 5. Exploratory analyses and their purpose.

Analysis	Purpose
Dimensions, data types, missing values, duplicates, descriptive statistics	Establish a data-quality baseline; exposed the duplicate rows and non-finite rate values.
Class-distribution plots	Demonstrate the imbalance between Normal (8.36%) and Anomaly (91.64%) traffic.
Histograms and distribution plots	Inspect the distribution of numerical features.
Box plots	Identify extreme observations and compare feature behavior across classes.
Correlation analysis and heatmaps	Investigate relationships between numerical variables and flag potentially redundant features.
Scatter plots	Compare selected feature pairs where a visual comparison was meaningful.

The most important practical finding was the severe class imbalance: Anomaly accounts for 91.64% of unique records. This motivated the use of Macro-F1, balanced accuracy, per-class metrics, ROC-AUC, PR-AUC, and confusion matrices instead of treating raw accuracy as sufficient. EDA also reinforced the need to handle non-finite rate variables and duplicate observations before any model comparison. Because of the dataset's size, sampling was used for computationally expensive visualizations where appropriate. EDA was used to understand the data and guide preprocessing and feature-selection decisions, not to select a model from visual patterns.

3.6 Feature Engineering and Feature Selection

Feature optimization is a central component of the proposed lightweight system. The original dataset contains a relatively large number of network-flow attributes, but not every variable is suitable or necessary for prediction. The columns in Table 6 were excluded from the predictive feature matrix; the remaining network-flow attributes were used as candidate features.

Table 6. Columns excluded from the predictive feature matrix.

Column	Role	Reason for exclusion
Label	Target variable	Is the prediction target
Cat	Category metadata	Alternate description of the target
Sub_Cat	Sub-category metadata	Alternate description of the target
Flow_ID	Flow identifier	High-cardinality identifier; enables memorization
Src_IP	Source IP identifier	High-cardinality identifier; device-specific
Dst_IP	Destination IP identifier	High-cardinality identifier; device-specific
Timestamp	Timestamp / metadata	Ties samples to specific capture moments

Feature importance was not delegated to a single algorithm. Four training-only criteria were computed independently: ANOVA F-test, mutual information, Random Forest impurity importance, and XGBoost importance. Their normalized scores were averaged to form a consensus ranking. Fixed top-10, top-15, top-20, top-30, and full 72-feature sets were then saved and reused consistently by the downstream model notebooks. This prevents each model from silently redefining a favorable subset and gives a fairer comparison of feature-count behavior, and it exposes the trade-off between reducing computational complexity and maintaining predictive performance. The Random Forest model selected the 10-feature subset, whereas Logistic Regression and XGBoost selected all 72 features on validation Macro-F1. Table 7 lists the ten highest-ranked features and Figure 3 shows the top 20. This feature-optimization approach is consistent with recent IoT intrusion-detection research, in which reducing unnecessary attributes is used to improve efficiency and support lightweight deployment [1], [10]–[17].

Table 7. Top ten features in the consensus ranking (training data only).

Rank	Feature	Interpretive role
1	Dst_Port	Destination service/endpoint behavior
2	Src_Port	Source-side connection behavior
3	ACK_Flag_Cnt	TCP acknowledgement pattern
4	Flow_Duration	Session temporal extent
5	Bwd_IAT_Std	Backward inter-arrival variability
6	Init_Bwd_Win_Byts	Initial backward TCP window
7	Flow_Pkts/s	Packet-rate intensity
8	Flow_IAT_Max	Maximum flow inter-arrival time
9	Idle_Mean	Average idle-period length
10	Flow_IAT_Mean	Mean flow inter-arrival time

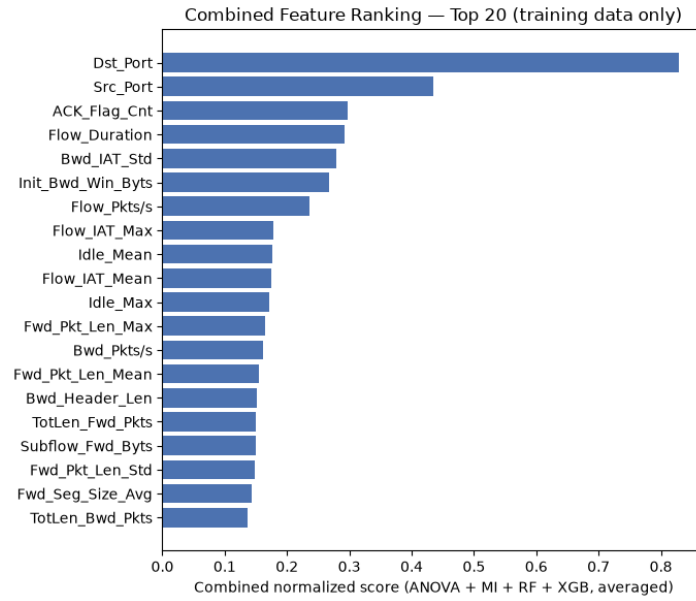


Figure 3. Consensus feature ranking (top 20) computed from training data only.

3.7 Model Building and Training

Three supervised classification algorithms were developed and compared on the same fixed feature subsets, with class-aware learning applied where supported. Model decisions were frozen before the final test evaluation, and the trained models were saved with Joblib so that final evaluation and explainability could reload them without retraining.

3.7.1 Logistic Regression

Logistic Regression was used as the transparent linear baseline. It models the probability of the binary target and shows how well a simple linear decision boundary performs on the network-flow features. Validation experiments compared different feature counts and selected the full 72-feature representation.

3.7.2 Random Forest

Random Forest [19] was used as a tree-based ensemble that combines many decision trees through bagging and can model non-linear relationships and interactions between network-flow variables. It was evaluated on the selected feature subsets, and the final configuration uses the top-10 subset.

3.7.3 XGBoost

XGBoost [20] was used as a regularized gradient-boosting alternative. Unlike Random Forest, where trees are built largely independently and combined by bagging, XGBoost builds trees sequentially so that each new tree can correct errors made by earlier ones. The baseline configuration used `n_estimators = 300`, `max_depth = 6`, `learning_rate = 0.1`, `tree_method = "hist"`, `random_state = 42`, `n_jobs = -1`, and `eval_metric = "logloss"`. Because the dataset contains far more Anomaly than Normal observations, class-aware sample weighting was used during training; this lets the objective account for imbalance without changing the validation and test distributions. XGBoost selected all 72 processed features.

Supporting notebooks also investigate from-scratch batch versus mini-batch gradient descent and L1/L2/ElasticNet regularization to study optimization and sparsity behavior.

3.8 Hyperparameter Tuning

Hyperparameter tuning was performed after the initial model and feature-count experiments, with the aim of improving validation performance while keeping computational cost practical. For XGBoost, a manually

bounded set of candidate configurations was evaluated instead of a very large search, because the dataset is large and unrestricted nested parallelism can increase computational overhead in a Jupyter/Windows environment. Macro-F1 on the validation set was the main selection criterion, because accuracy alone can hide differences in minority-class performance. Table 8 gives the final configuration of each model and Table 9 compares the XGBoost baseline with its tuned values.

Table 8. Final selected configuration of each model.

Model	Features	Selected hyperparameters
Logistic Regression	72	C = 10, lbfgs solver, balanced class weights
Random Forest	Top 10	200 trees, maximum depth 30, all selected features considered at each split, minimum split size 2, minimum leaf size 1, balanced_subsample class weighting
XGBoost	72	200 trees, maximum depth 8, learning rate 0.2, row subsample 0.9, column subsample 0.6, minimum child weight 1, L1 penalty 0, L2 penalty 2.0, random state 42

Table 9. XGBoost hyperparameters: baseline versus final tuned configuration.

Parameter	Baseline	Final (tuned)
n_estimators	300	200
max_depth	6	8
learning_rate	0.1	0.2
subsample	—	0.9
colsample_bytree	—	0.6
min_child_weight	—	1
reg_alpha	—	0
reg_lambda	—	2.0
random_state	42	42

— = not specified for the baseline configuration.

The feature-count experiment for XGBoost, which preceded hyperparameter tuning, produced the validation Macro-F1 values in Table 10 and Figure 4. Validation Macro-F1 rose from 0.995246 with 10 features to 0.996296 with all 72, a gain of only 0.001050, so even the 10-feature subset retained near-peak performance. The 72-feature configuration was retained because it had the highest validation Macro-F1; after tuning, this configuration reached a validation Macro-F1 of 0.996732.

Table 10. XGBoost validation Macro-F1 for each feature count.

Number of features	Validation Macro-F1
10	0.995246
15	0.996034
20	0.995946
30	0.996165
72	0.996296

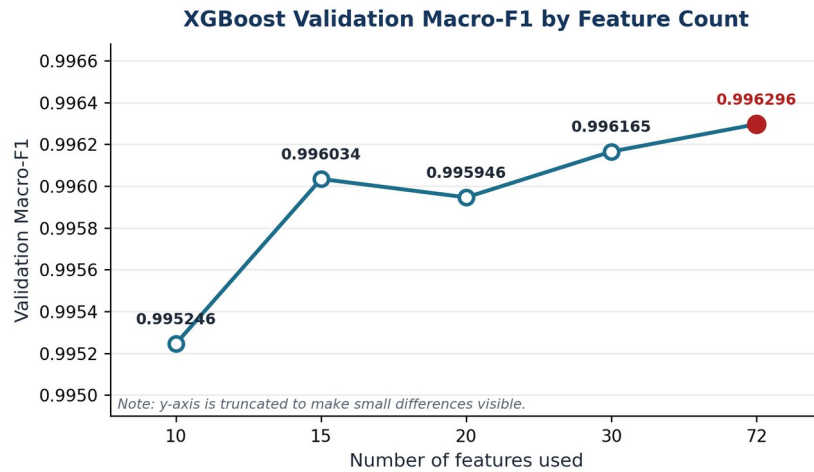


Figure 4. XGBoost validation Macro-F1 for 10, 15, 20, 30, and 72 features (y-axis truncated to show small differences).

3.9 Model Evaluation

The models were evaluated as binary classification systems using metrics suited to an imbalanced intrusion-detection problem. Macro-F1 is the primary selection metric because it gives equal weight to the Normal and Anomaly F1 scores despite the severe class imbalance; the other measures in Table 11 support it. The final test set contains 92,340 flows and is evaluated only after the three candidate models are frozen.

Table 11. Evaluation metrics and their role in this study.

Metric	Definition	Role in this study
Accuracy	Share of all observations classified correctly.	Reported, but not sufficient alone given the 91.64% Anomaly share.
Precision	Share of predicted anomalies that are truly anomalies.	Indicates the number of false alarms.
Recall	Share of actual anomalies that are detected.	Critical for intrusion detection: missed attacks are false negatives.
F1-score / Macro-F1	Harmonic mean of precision and recall; Macro-F1 averages the class-wise F1 scores.	Primary model-selection metric.
Balanced accuracy	Average of the recall on the Normal and Anomaly classes.	Guards against the dominant class hiding errors.
ROC-AUC	Ability to separate the two classes across all classification thresholds.	Threshold-independent separation measure.
PR-AUC	Area under the precision–recall curve.	Informative under strong class imbalance.
Confusion matrix	Counts of TP, TN, FP, and FN.	Shows exactly which errors each model makes.

For the positive Anomaly class:

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

Here TP (true positives) are anomalies correctly detected, TN (true negatives) are Normal flows correctly identified, FP (false positives) are Normal flows incorrectly classified as Anomaly, and FN (false negatives) are anomalies incorrectly classified as Normal. Finally, SHAP-based explainability was applied to the frozen trained model to examine which network-flow features contributed to its predictions, providing global feature importance and individual prediction explanations, which connects the predictive component to the project's goal of explainable intrusion detection.

4. Results and Discussion

4.1 Preprocessing Results

The original IoTID20 file contained 625,783 records and 86 columns. During the initial inspection, 164,087 exact duplicate rows (26.22% of the raw dataset) were identified and removed, leaving 461,696 unique records. The malformed destination-port field was corrected, non-finite flow-rate values were replaced with missing values for training-fitted imputation, and constant predictors were eliminated. Seven target-related, identifier, or metadata columns (Label, Cat, Sub_Cat, Flow_ID, Src_IP, Dst_IP, and Timestamp) were excluded from the feature matrix; together with the constant predictors removed in the audit, this left 72 predictive network-flow features.

The dataset is highly imbalanced, with 91.64% Anomaly and 8.36% Normal records, so accuracy was not treated as sufficient on its own, and class-sensitive metrics such as Macro-F1, precision, recall, and ROC-AUC were used. The split contains 295,484 training rows, 73,872 validation rows, and 92,340 untouched final-test rows, and the test split was created before learned imputation, scaling, feature ranking, model selection, or hyperparameter tuning (see Table 3). The feature-selection experiments then compared 10, 15, 20, 30, and all 72 features to examine the trade-off between model compactness and predictive performance.

4.2 Model Performance

Table 12 reports the final untouched test-set results for the three candidate models. The best value in each column is shown in bold.

Table 12. Final test-set performance of the candidate models (Macro-F1 is the primary metric).

Model / Study	Accuracy	Macro Precision	Macro Recall	Macro-F1	ROC-AUC
Logistic Regression (72 features)	0.9141	0.7428	0.9200	0.7974	0.9747
Random Forest (Top 10)	0.9993	0.9993	0.9963	0.9978	0.9996
XGBoost (72 features)	0.9988	0.9962	0.9957	0.9960	0.9999
El Mahfoud et al. [1] – DT (context only)	0.9980	NR	NR	NR	NR

Note: NR = not reported in the cited study. The external study is shown for context only; direct ranking is not valid because datasets, preprocessing, target definitions, and splits differ.

BEST TEST MACRO-F1 0.9978 <i>Random Forest, 10 features</i>	COMPACT ALTERNATIVE 0.921 MB <i>XGBoost serialized model</i>	FINAL TEST SET 92,340 <i>Untouched network flows</i>
---	--	--

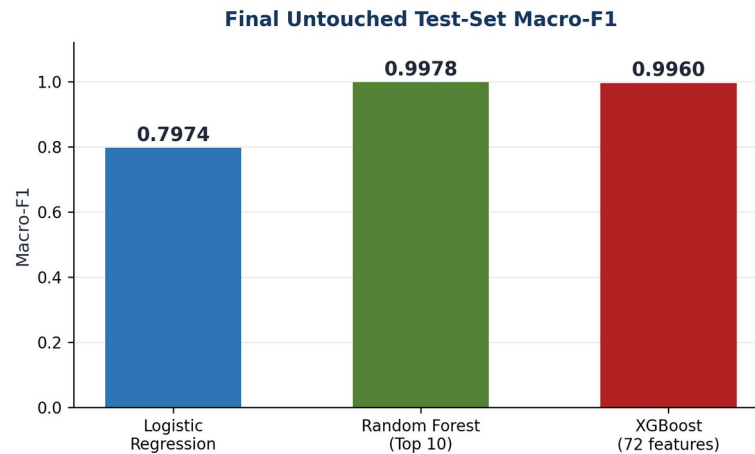


Figure 5. Final untouched test-set Macro-F1. Random Forest achieves the strongest class-balanced score, with XGBoost very close behind.

Random Forest achieved the highest final Macro-F1 (0.997803) and 99.93% accuracy using only the top 10 features. On the Normal class, precision, recall, and F1 were approximately 0.9992, 0.9927, and 0.9960; on the Anomaly class they were approximately 0.9993, 0.9999, and 0.9996. This confirms that the model performed strongly on both classes rather than obtaining a high accuracy primarily from the dominant Anomaly class.

XGBoost achieved a slightly lower Macro-F1 of 0.995970 but the highest ROC-AUC (0.999943) and PR-AUC (0.999995). Its serialized model size was 0.921 MB compared with 16.158 MB for Random Forest, and its measured final-test inference time was 0.0513 s versus 0.1325 s. These measurements are machine-dependent, but they reveal an important deployment trade-off: the strongest Macro-F1 model is not automatically the smallest or fastest model.

Logistic Regression reached 91.41% accuracy but only 0.797375 Macro-F1. Its weaker performance on the minority Normal class demonstrates why accuracy alone is unsuitable for this imbalanced security task and suggests that non-linear feature interactions are important in the IoT flow representation.

Table 13 compares the validation Macro-F1 used for model selection with the final test Macro-F1. The two agree closely for every model, and the ranking is identical on both splits.

Table 13. Macro-F1 on the validation set (model selection) and on the untouched test set.

Model	Validation Macro-F1	Test Macro-F1	Difference (test – validation)
Logistic Regression	0.7967	0.7974	+0.0007
Random Forest (Top 10)	0.9986	0.9978	–0.0008
XGBoost (72 features)	0.9967	0.9960	–0.0007

KEY FINDING

Test scores differ from validation scores by at most 0.0008 Macro-F1 for every model. Selecting configurations on validation data therefore did not overfit to it, and the untouched test set confirms the validation ranking: Random Forest first, XGBoost a close second, Logistic Regression far behind.

4.3 Visual Results and Discussion

4.3.1 Confusion matrix

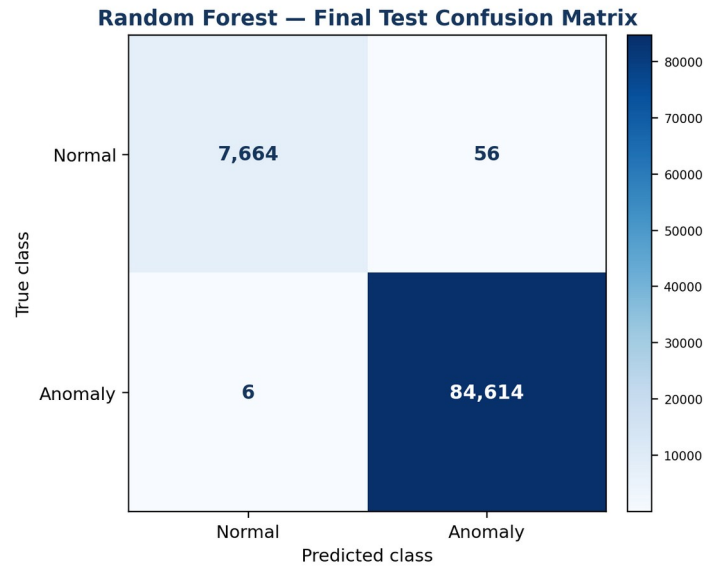


Figure 6. Final-test confusion matrix for the top-10 Random Forest: 7,664 Normal and 84,614 Anomaly flows were correctly classified, with 62 total errors.

The final Random Forest confusion matrix corresponds to 56 false positives (Normal flows marked as Anomaly) and only 6 false negatives (Anomaly flows marked as Normal). For intrusion detection, the very low false-negative count is especially valuable because missed malicious activity can carry a higher operational cost than an additional investigation. The remaining false positives still matter because excessive alerts can create analyst fatigue; threshold calibration can therefore be explored in future deployments. On the validation set, XGBoost reached an overall accuracy of 99.8998% with approximately 99.42% recall for Normal traffic and 99.94% recall for Anomaly traffic, showing balanced behavior across both classes.

4.3.2 ROC-AUC

ROC-AUC measures how well a model separates Normal from Anomaly traffic over all classification thresholds. XGBoost achieved a ROC-AUC of 0.999889 on the validation data and 0.999943 on the test set, the highest of the three models on the test set (Random Forest 0.9996, Logistic Regression 0.9747). Values this close to 1 mean the model keeps a high true-positive rate while holding the false-positive rate low across thresholds.

4.3.3 Precision–Recall AUC

The precision–recall curve is particularly relevant because the classes are unequally distributed. XGBoost achieved a PR-AUC of 0.999989 on the validation data and 0.999995 on the test set, which shows that it kept both precision and recall high across thresholds. This matters for intrusion detection because both missed attacks and false alarms carry a cost. Table 14 collects the full XGBoost metrics on both splits.

Table 14. XGBoost metrics on the validation set and on the untouched test set.

Metric	Validation	Test
Accuracy	0.9990 (99.8998%)	0.9988
Macro precision	0.9967	0.9962
Macro recall	0.9968	0.9957
Macro-F1	0.9967	0.9960
ROC-AUC	0.999889	0.999943
PR-AUC	0.999989	0.999995

4.3.4 Feature importance

Unlike the Logistic Regression and Random Forest experiments, XGBoost was trained on all 72 candidate predictive features. Its results indicate that the model was able to use information from the broader network-flow feature set to separate Normal from Anomaly traffic, although the feature-count experiment in Section 3.8 shows that a much smaller subset would already have performed almost as well. Feature importance should be read as a measure of model reliance rather than proof that a feature independently causes an intrusion.

4.3.5 SHAP explainability

SHAP analysis was performed on the frozen Random Forest model selected from the validation comparison, using a reproducible sample of 3,000 validation records. SHAP values describe the direction and magnitude of each feature's effect on individual predictions: positive values push a prediction toward the Anomaly class and negative values toward the Normal class. Table 15 lists the ten features by mean absolute SHAP value and Figure 7 plots them.

Table 15. Features ranked by mean absolute SHAP value (Random Forest, 3,000 validation records).

Rank	Feature	Mean SHAP
1	Dst_Port	0.290043
2	Init_Bwd_Win_Byts	0.095813
3	Src_Port	0.057591
4	Flow_Duration	0.038285
5	ACK_Flag_Cnt	0.021214
6	Bwd_IAT_Std	0.016969
7	Idle_Mean	0.005668
8	Flow_Pkts/s	0.003600
9	Flow_IAT_Mean	0.001980
10	Flow_IAT_Max	0.001359

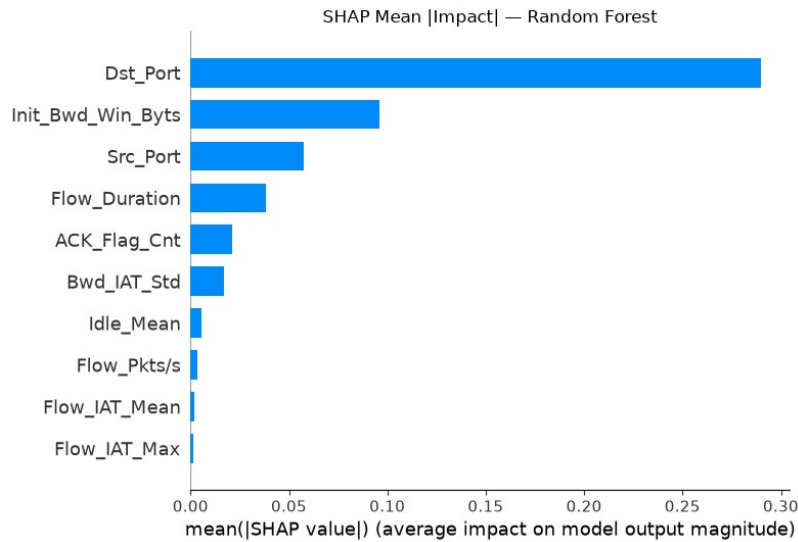


Figure 7. SHAP mean absolute impact for the final Random Forest candidate.

`Dst_Port` is the dominant contributor, with a mean absolute SHAP value (0.290) about three times that of the runner-up, `Init_Bwd_Win_Byts` (0.096). Because the Random Forest was trained on only the ten top-ranked features, SHAP necessarily ranks those same ten, so the overlap with the consensus ranking in Figure 3 is partly by construction. The informative comparison is the ordering: the same six features occupy the top six places in both rankings, but in a different order (for example, `Init_Bwd_Win_Byts` is second under SHAP and sixth in the consensus ranking). This suggests that the compact model relies on a small set of network-flow characteristics rather than a large collection of weak variables. SHAP explains the fitted model, not the underlying cause of an attack, so these values should not be read as proof that a feature causes an intrusion.

4.4 Discussion of Results

All three approaches identified network-intrusion patterns, but their performance differed considerably. On the validation set, Logistic Regression reached a Macro-F1 of about 0.7967, while Random Forest and XGBoost reached about 0.9986 and 0.9967. The strong results of the tree-based models are consistent with their ability to capture non-linear relationships and interactions among network-flow features, which matters because attack behavior is rarely captured well by simple linear boundaries.

Random Forest achieved a slightly higher Macro-F1 than XGBoost, but the two models have different resource profiles. Random Forest needed only 10 selected features but its saved model is about 16.158 MB, whereas XGBoost used all 72 features in a model of about 0.921 MB with faster measured inference. Model performance should therefore be weighed together with feature count, model size, computational requirements, and the intended deployment environment. Random Forest is lightweight in input dimensionality, while XGBoost is lightweight in serialized size and measured inference time; the appropriate production model depends on the deployment bottleneck, whether that is feature-collection complexity, RAM/storage, latency, false-alarm tolerance, or analyst interpretability.

LIMITATIONS AND SCOPE

The data come from a controlled smart-home testbed with a limited number of devices and attack types, so performance on larger real-world IoT networks has not been demonstrated.

Port features dominate the explanations (`Dst_Port` and `Src_Port`). Ports can reflect how the testbed was configured, so a model may partly learn the testbed rather than general attack behavior. Cross-dataset and grouped or temporal evaluation are needed to test this.

Removing exact duplicates does not remove near-duplicate flows from the same session, which can still make a random split optimistic.

Inference times are single-environment measurements, and SHAP values show model attribution, not causation.

5. Conclusion

This project addressed the detection of anomalous network traffic in IoT environments with a lightweight, feature-optimized, and explainable machine-learning approach. The workflow began with a detailed data-quality audit, removed exact duplicates, corrected malformed and non-finite values, controlled identifier and target leakage, isolated the final test set before learned preprocessing, and produced a consistent 72-feature representation. A four-method consensus ranking then identified compact feature subsets, after which Logistic Regression, Random Forest, and XGBoost were tuned on validation data only and evaluated once on the untouched test set.

KEY FINDINGS

- 164,087 exact duplicates (26.22%) were removed, leaving 461,696 unique flows and 72 predictors.
- Random Forest on the top 10 features reached a test Macro-F1 of **0.997803** and 99.93% accuracy, with only 6 false negatives and 56 false positives among 92,340 flows.
- XGBoost delivered very similar quality (Macro-F1 0.995970) with a **0.921 MB** model, against 16.158 MB for Random Forest, and faster measured inference.
- Logistic Regression reached only 0.797375 Macro-F1, so non-linear models are needed for this task.
- SHAP ranked `Dst_Port`, `Init_Bwd_Win_Byts`, and `Src_Port` as the strongest contributors to the Random Forest's predictions.

Test results matched validation results closely for all three models, which supports the soundness of the validation-based selection procedure. The project also highlights a practical trade-off between predictive performance, feature requirements, model size, and interpretability: Random Forest gave the best score with a compact feature set, while XGBoost gave similarly high performance with a much smaller saved model. The choice for an actual IoT deployment therefore depends on the available computational resources, feature availability, and operational requirements.

Overall, the results demonstrate that careful experimental design, feature optimization, rigorous validation, and SHAP explainability can be as important as the choice of classifier when building a trustworthy IoT intrusion-detection system. Because the dataset was created in a controlled environment with a limited number of devices and attacks, further testing on larger real-world IoT networks is required before the system is deployed.

6. Future Work

- **Validate on larger, more diverse, and external datasets.** Evaluate the approach on IoT datasets with traffic from different devices, network environments, and attack categories, including cross-dataset testing to measure true domain generalization and to expose dataset-specific limitations.
- **Extend the classification task.** Move from binary Normal/Anomaly prediction to attack-category and attack-subcategory classification.
- **Use stricter splits.** Apply temporal, device-grouped, or capture-session-grouped splits to reduce environment-specific correlation between training and test traffic.
- **Calibrate decision thresholds.** Trade false negatives against false positives explicitly, according to the operational cost of missed attacks and alert fatigue.
- **Evaluate additional algorithms and ensembles.** Compare further machine-learning and deep-learning models, and study whether combining different models improves detection.
- **Perform deeper hyperparameter optimization.** Replace the bounded search with Bayesian optimization, randomized search, or automated optimization to explore a wider parameter space.
- **Investigate explainable AI techniques.** Complement SHAP with LIME, permutation-based explanations, and feature-interaction analysis, and present them in analyst-facing dashboards.
- **Evaluate robustness and drift.** Monitor data drift and explanation drift, study robustness under changing traffic distributions, and investigate online or incremental learning to decide when retraining or feature re-selection is needed.
- **Develop a deployable real-time, cloud, or edge application.** Integrate the IDS into a monitoring system that processes network flows and raises alerts, and benchmark model compression, pruning, quantization, and TinyML deployment on constrained gateways or microcontrollers.
- **Study resource requirements.** Measure inference time, memory usage, CPU utilization, and throughput to judge whether the selected model can run on constrained IoT or edge devices.
- **Investigate adaptive feature selection.** Examine methods that identify the most useful features automatically as traffic patterns evolve.
- **Develop explainable security alerts.** Attach to each detected anomaly a concise explanation of the network-flow features that contributed most to the prediction.

7. References

- [1] E. El Mahfoud, S. El Hajla, Y. Maleh, and S. Mounir, "Machine Learning Algorithms for Intrusion Detection in IoT Prediction and Performance Analysis," *Procedia Computer Science*, vol. 236, pp. 460–467, 2024, doi: 10.1016/j.procs.2024.05.054.
- [2] M. M. Inuwa and R. Das, "A comparative analysis of various machine learning methods for anomaly detection in cyber attacks on IoT networks," *Internet of Things*, vol. 26, Art. no. 101162, 2024, doi: 10.1016/j.iot.2024.101162.
- [3] S. Bin Hulayyil, S. Li, and N. Saxena, "Explainable AI-based intrusion detection in IoT systems," *Internet of Things*, vol. 31, Art. no. 101589, 2025, doi: 10.1016/j.iot.2025.101589.
- [4] Y. Wang, M. A. Azad, M. Zafar, and A. Gul, "Enhancing AI transparency in IoT intrusion detection using explainable AI techniques," *Internet of Things*, vol. 33, Art. no. 101714, 2025, doi: 10.1016/j.iot.2025.101714.
- [5] S. Sadhwani, A. Navare, A. Mohan, R. Muthalagu, and P. M. Pawar, "IoT-based intrusion detection system using explainable multi-class deep learning approaches," *Computers and Electrical Engineering*, vol. 123, Art. no. 110256, 2025, doi: 10.1016/j.compeleceng.2025.110256.

- [6] K. Yang, J. Wang, G. Zhao, X. Wang, W. Cong, M. Yuan, J. Luo, X. Dong, J. Wang, and J. Tao, "NIDS-CNNRF integrating CNN and random forest for efficient network intrusion detection model," *Internet of Things*, vol. 32, Art. no. 101607, 2025, doi: 10.1016/j.iot.2025.101607.
- [7] J. Ahmad, S. Latif, I. U. Khan, M. S. Alshehri, M. S. Khan, N. Alasbali, and W. Jiang, "An interpretable deep learning framework for intrusion detection in industrial Internet of Things," *Internet of Things*, vol. 33, Art. no. 101681, 2025, doi: 10.1016/j.iot.2025.101681.
- [8] A. Hamdouchi and A. Idri, "Evaluating the performance of TinyML singular and ensemble techniques for intrusion detection in IoT networks," *Microprocessors and Microsystems*, vol. 117, Art. no. 105172, 2025, doi: 10.1016/j.micpro.2025.105172.
- [9] T. Hasan and S. Tasnim, "Real-time explainable IoT security with machine learning and CTGAN-enhanced detection for resource-constrained devices," *Ad Hoc Networks*, vol. 178, Art. no. 103937, 2025, doi: 10.1016/j.adhoc.2025.103937.
- [10] F. Ebrahimi, R. Javidan, R. Akbari, and Y. Hosseini, "Intrusion detection in the Internet of Things using convolutional neural networks: an explainable AI approach," *Cybersecurity*, vol. 8, Art. no. 66, 2025, doi: 10.1186/s42400-025-00369-2.
- [11] J. Azimjonov and T. Kim, "Designing accurate lightweight intrusion detection systems for IoT networks using fine-tuned linear SVM and feature selectors," *Computers & Security*, vol. 137, Art. no. 103598, 2024, doi: 10.1016/j.cose.2023.103598.
- [12] J. García, J. Entrena, and Á. Alesanco, "Empirical evaluation of feature selection methods for machine learning based intrusion detection in IoT scenarios," *Internet of Things*, vol. 28, Art. no. 101367, 2024, doi: 10.1016/j.iot.2024.101367.
- [13] M. M. Alani, "An explainable efficient flow-based Industrial IoT intrusion detection system," *Computers & Electrical Engineering*, vol. 108, Art. no. 108732, 2023, doi: 10.1016/j.compeleceng.2023.108732.
- [14] S. D. A. Rihan, M. Anbar, and B. A. Alabsi, "Approach for detecting attacks on IoT networks based on ensemble feature selection and deep learning models," *Sensors*, vol. 23, no. 17, Art. no. 7342, 2023, doi: 10.3390/s23177342.
- [15] X. Chen, M. Liu, Z. Wang, and Y. Wang, "Explainable deep learning-based feature selection and intrusion detection method on the Internet of Things," *Sensors*, vol. 24, no. 16, Art. no. 5223, 2024, doi: 10.3390/s24165223.
- [16] M. B. Musthafa, S. Huda, Y. Kodera, M. A. Ali, S. Araki, J. Mwaura, and Y. Nogami, "Optimizing IoT intrusion detection using balanced class distribution, feature selection, and ensemble machine learning techniques," *Sensors*, vol. 24, no. 13, Art. no. 4293, 2024, doi: 10.3390/s24134293.
- [17] M. Almohaimeed and F. Albalwy, "Enhancing IoT network security using feature selection for intrusion detection systems," *Applied Sciences*, vol. 14, no. 24, Art. no. 11966, 2024, doi: 10.3390/app142411966.
- [18] I. Ullah and Q. H. Mahmoud, "A Scheme for Generating a Dataset for Anomalous Activity Detection in IoT Networks," in *Advances in Artificial Intelligence*, LNCS 12109, Springer, 2020, pp. 508–520, doi: 10.1007/978-3-030-47358-7_52.
- [19] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [20] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [21] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

Signature of the Guide

Course Instructor